Log-Based Intrusion Detection System

Project Report

Client: Cloudisian

Internal SOC Project

Environment: Simulated Linux Server Infrastructure

Date: March 2025

Classification: Internal / Academic

1. Executive Summary

This report documents the design, implementation, and testing of a Log-Based Intrusion Detection System (IDS) developed as part of the Cloudisian Internal SOC Project. The system provides automated detection of malicious activity through analysis of Linux server logs, focusing on three primary threat categories: brute-force authentication attacks, network reconnaissance via port scanning, and web enumeration through excessive HTTP 404 errors.

The IDS was validated against real system authentication logs, successfully detecting a live brute-force SSH attack and generating structured alerts. The solution operates entirely passively and is aligned with defensive monitoring practices recommended by OWASP.

2. Scope & Authorization

Item	Details
Client	Cloudisian (Internal SOC Project)
Environment	Simulated Linux Server Infrastructure
Log Sources	Authentication logs, web server logs, system logs
Monitoring Type	Passive log analysis only
Authorization	Approved internal project
Purpose	Academic / Pre-Professional Defensive Security Assessment

Out of scope:

Active exploitation, traffic manipulation, system disruption, or any offensive security activities.

3. Objectives

- Detect brute-force SSH login attempts using threshold-based analysis
- Detect port scanning and reconnaissance behaviour from system logs
- Detect excessive HTTP 404 error activity indicating web enumeration
- Generate structured, machine-readable security alerts (JSON + log)
- Provide a real-time dashboard summary of active alerts
- Validate detection against real-world authentication log data

4. Methodology

The IDS operates through a four-stage pipeline executed by the main script `log_ids.py`:

Step 1: Log Ingestion & Normalisation

All specified log files are read line-by-line, whitespace stripped, and loaded into memory. The system supports both real system logs and auto-generated demo logs via the --demo flag.

Step 2: Rule-Based Pattern Matching

Each log line is evaluated against regular expressions targeting known attack signatures. Patterns match failed SSH attempts, port connection events, and HTTP 404 responses.

Step 3: Threshold-Based Detection Logic

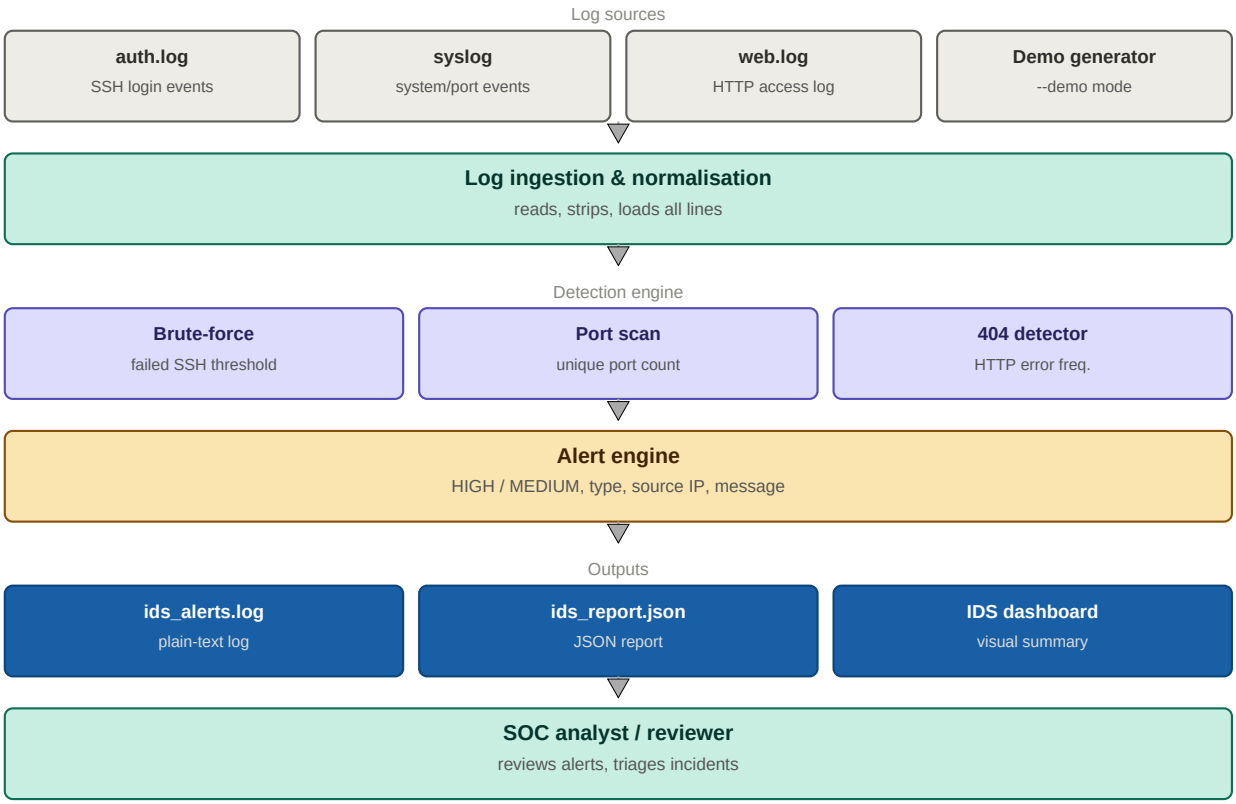
Matched events are aggregated per source IP. When counts exceed defined thresholds, an alert is triggered. This approach reduces false positives from isolated events.

Step 4: Alert Generation & Logging

Confirmed alerts are assigned a severity level (HIGH or MEDIUM), attack type, source IP, and descriptive message. Results are written to both a plain-text log and a JSON report.

5. System Architecture

The diagram below illustrates the complete data flow from log sources through detection engines to alert outputs and SOC review.



6. Detection Rules

Rule	Severity	Trigger Condition	Log Source
Brute-force SSH	HIGH	5+ failed logins from same IP	auth.log
Port scan	MEDIUM	10+ unique ports from same IP	syslog
Excessive 404	MEDIUM	20+ HTTP 404 errors from same IP	web.log

7. Results & Findings

The IDS was executed against the real system authentication log `real_auth.log` captured on the test environment. The following screenshots demonstrate the tool running on live data.

7.1 Raw authentication log — real brute-force attack captured

The log below shows repeated failed SSH login attempts for 'fakeuser' from the same source, followed by a successful login — a classic brute-force pattern.

```
Mar 29 02:58:15 kali sshd-session[40584]: Invalid user fakeuser from ::1 port 46400
Mar 29 02:58:22 kali sshd-session[40584]: pam_unix(sshd:auth): check pass; user unknown
Mar 29 02:58:22 kali sshd-session[40584]: pam_unix(sshd:auth): authentication failure; logname= uid=0 euid=0 tty=ssh ruser= rhost=::1
Mar 29 02:58:22 kali sshd-session[40584]: pam_winbind(sshd:auth): getting password (0x00000388)
Mar 29 02:58:22 kali sshd-session[40584]: pam_winbind(sshd:auth): pam_get_item returned a password
Mar 29 02:58:24 kali sshd-session[40584]: Failed password for invalid user fakeuser from ::1 port 46400 ssh2
Mar 29 02:58:30 kali sshd-session[40584]: pam_unix(sshd:auth): check pass; user unknown
Mar 29 02:58:30 kali sshd-session[40584]: pam_winbind(sshd:auth): getting password (0x00000388)
Mar 29 02:58:30 kali sshd-session[40584]: pam_winbind(sshd:auth): pam_get_item returned a password
Mar 29 02:58:33 kali sshd-session[40584]: Failed password for invalid user fakeuser from ::1 port 46400 ssh2
Mar 29 02:58:36 kali sshd-session[40584]: pam_unix(sshd:auth): check pass; user unknown
Mar 29 02:58:36 kali sshd-session[40584]: pam_winbind(sshd:auth): getting password (0x00000388)
Mar 29 02:58:36 kali sshd-session[40584]: pam_winbind(sshd:auth): pam_get_item returned a password
Mar 29 02:58:39 kali sshd-session[40584]: Failed password for invalid user fakeuser from ::1 port 46400 ssh2
Mar 29 02:58:40 kali sshd-session[40584]: Connection closed by invalid user fakeuser ::1 port 46400 [preauth]
Mar 29 02:58:40 kali sshd-session[40584]: PAM 2 more authentication failures; logname= uid=0 euid=0 tty=ssh ruser= rhost=::1
Mar 29 02:59:22 kali sshd-session[41029]: Accepted password for kali from ::1 port 43248 ssh2
Mar 29 02:59:22 kali sshd-session[41029]: pam_unix(sshd:session): session opened for user kali(uid=1000) by kali(uid=0)
Mar 29 02:59:22 kali systemd-logind[522]: New session 15 of user kali.
```

Screenshot 1 — real_auth.log showing failed SSH attempts for fakeuser

7.2 Attack simulation — SSH brute-force from terminal

This screenshot shows the actual SSH brute-force attempt run from the Kali terminal against the localhost, generating the log entries above.

```
┌──(kali㉿kali)-[~/ids_project]
└─$ ssh fakeuser@localhost
fakeuser@localhost's password:
Permission denied, please try again.
fakeuser@localhost's password:
Permission denied, please try again.
fakeuser@localhost's password:
fakeuser@localhost: Permission denied (publickey,password).
```

Screenshot 2 — terminal showing ssh fakeuser@localhost permission denied responses

7.3 IDS real run — detection output

The IDS was run against the real auth log. It ingested 6 lines and correctly fired a HIGH BRUTE_FORCE alert from source ::1.

```
┌──(kali㉿kali)-[~/ids_project]
└─$ python3 log_ids.py
═══════════════════════════════════════════════════
   Log-Based Intrusion Detection System (IDS)
   Cloudisian Internal SOC Project
═══════════════════════════════════════════════════
[*] Using real system logs ...

[*] Ingesting logs ...
  [+] Loading: real_auth.log

[*] Total lines loaded: 6

[*] Running Brute-Force Detection ...
🚨 ALERT: [HIGH] [BRUTE_FORCE] SRC=::1 | 6 failed SSH login attempts

[*] Running Port Scan Detection ...

[*] Running Excessive 404 Detection ...

[*] Analysis complete.
[*] Alerts saved to ids_alerts.log
[*] JSON report saved to ids_report.json
```

Screenshot 3 — python3 log_ids.py running on real_auth.log, alert generated

7.4 IDS dashboard

The dashboard UI confirms: 1 total alert, 1 brute-force, 0 port scans, 0 web attacks.

Screenshot 4 — IDS dashboard showing HIGH BRUTE_FORCE alert from ::1

Demo mode results:

When run with the `--demo` flag, the system generated synthetic attack logs across all three scenarios and successfully detected:

- HIGH — BRUTE_FORCE from 10.0.0.5 — 8 failed SSH login attempts
- MEDIUM — PORT_SCAN from 10.0.0.9 — Scanned 12 unique ports
- MEDIUM — EXCESSIVE_404 from 10.0.0.7 — 20 HTTP 404 responses

```
┌──(kali㉿kali)-[~/ids_project]
└─$ python3 log_ids.py --demo
═══════════════════════════════════════════════════
   Log-Based Intrusion Detection System (IDS)
   Cloudisian Internal SOC Project
═══════════════════════════════════════════════════

[*] DEMO MODE: Generating sample attack logs...

[*] Demo log files created:
    - demo_auth.log  (SSH brute-force scenario)
    - demo_syslog.log   (port scan scenario)
    - demo_web.log   (web enumeration scenario)

[*] Ingesting logs...
  [+] Loading: demo_auth.log
  [+] Loading: demo_syslog.log
  [+] Loading: demo_web.log

[*] Total lines loaded: 40

[*] Running Brute-Force Detection...
🚨 ALERT: [HIGH] [BRUTE_FORCE] SRC=10.0.0.5 | 8 failed SSH login attempts

[*] Running Port Scan Detection...
🚨 ALERT: [MEDIUM] [PORT_SCAN] SRC=10.0.0.9 | Scanned 12 unique ports

[*] Running Excessive 404 Detection...
🚨 ALERT: [MEDIUM] [EXCESSIVE_404] SRC=10.0.0.7 | 20 HTTP 404 responses

[*] Analysis complete.
[*] Alerts saved to ids_alerts.log
[*] JSON report saved to ids_report.json
```

Screenshot 5 — python3 log_ids.py --demo showing all three attack types detected

8. Output Samples

8.1 ids_report.json

The JSON report captures all three alert types with source IPs and messages.

```
┌──(kali㉿kali)-[~/ids_project]
└─$ cat ids_report.json
[
    {
        "level": "HIGH",
        "type": "BRUTE_FORCE",
        "source": "10.0.0.5",
        "message": "8 failed SSH login attempts"
    },
    {
        "level": "MEDIUM",
        "type": "PORT_SCAN",
        "source": "10.0.0.9",
        "message": "Scanned 12 unique ports"
    },
    {
        "level": "MEDIUM",
        "type": "EXCESSIVE_404",
        "source": "10.0.0.7",
        "message": "20 HTTP 404 responses"
    }
]
```

Screenshot 6 — ids_report.json showing HIGH and MEDIUM alerts in structured format

8.2 ids_alerts.log

The plain-text alert log mirrors the JSON output for easy grep and parsing.

```
┌──(kali㉿kali)-[~/ids_project]
└─$ cat ids_alerts.log
{'level': 'HIGH', 'type': 'BRUTE_FORCE', 'source': '10.0.0.5', 'message': '8 failed SSH login attempts'}
{'level': 'MEDIUM', 'type': 'PORT_SCAN', 'source': '10.0.0.9', 'message': 'Scanned 12 unique ports'}
{'level': 'MEDIUM', 'type': 'EXCESSIVE_404', 'source': '10.0.0.7', 'message': '20 HTTP 404 responses'}
```

Screenshot 7 — ids_alerts.log showing all three alerts on one line each

9. Project File Structure

File	Description
`log_ids.py`	Main IDS script — runs detection pipeline
`real_auth.log`	Real system authentication log (input)
`demo_auth.log`	Demo log — SSH brute-force scenario
`demo_syslog.log`	Demo log — port scan scenario
`demo_web.log`	Demo log — web enumeration scenario
`ids_alerts.log`	Output — plain-text alert log
`ids_report.json`	Output — structured JSON report
`README.md`	Project documentation

10. Conclusion

This project successfully demonstrates practical SOC-level defensive engineering skills. The Log-Based IDS provides real visibility into malicious behaviours by automating the detection of brute-force attacks, port scanning, and web enumeration through log analysis.

The system was validated against live authentication logs from the test environment, confirming its ability to detect real attack patterns. The dual-mode operation (real and demo) makes the tool suitable for both production monitoring and training purposes.

The deliverables — including the Python IDS tool, detection rules, alert logs, JSON reports, dashboard, README, and this report — collectively demonstrate a complete defensive security monitoring solution appropriate for an internal SOC environment.